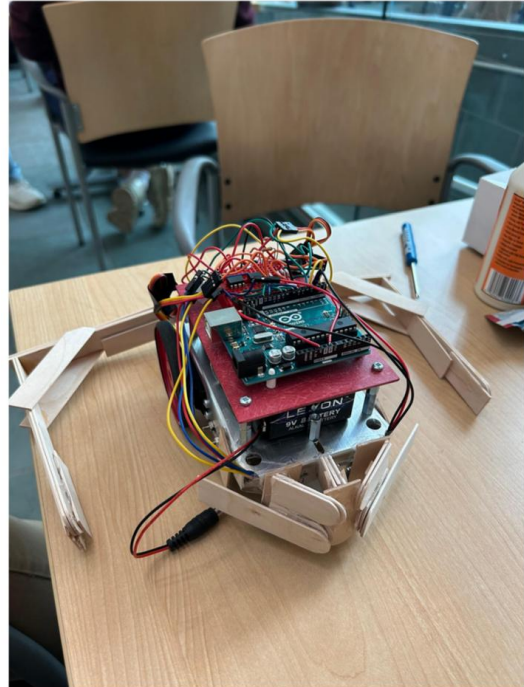


Group 53: Christopher Miranda Soto (cm2233), Hugo Larios (hdl45), Mathis Vigneau (mav222)

1. Robot Design and Strategy Overview

Our strategy for this competition was to use a wide-reaching grabbing mechanism to collect cubes efficiently, move through areas with the highest cube density, and stop after enough time had passed to gather at least half the cubes. The grabbing mechanism uses two servos to rotate popsicle-stick arms from an “up” starting position to a deployed “down” position for the duration of the round. We



*This is Frank. He's a little shy.
Say "Hi Frank!"*

chose popsicle sticks because they are lightweight, easy to repair, and highly adaptable. When deployed, the arms cover the maximum allowed area and guide cubes into the robot with the help of a front ram. The robot's electrical system includes an Arduino Uno, two servos, a 9V battery, wheel motors controlled by two H-bridges, a color sensor, and two QTI sensors that detect the black border (see Appendix B). The robot was controlled by custom Arduino code designed to maximize movement speed, follow efficient and adaptable paths, avoid obstacles such as other robots, and coordinate all electrical components (see Appendices D and E).

2. Design Process Reflection

At the start of the project, we met to discuss robot designs and initially chose a front-side harvesting system inspired by combine harvesters that would pull cubes into storage. After presenting this idea for the first milestone, we used the provided wheels, servos, and sensors to give the robot mobility while developing and testing code through repeated hardware and software troubleshooting. Building and programming were collaborative efforts, with the team working together to solve issues. Our biggest challenge was a lack of proper materials caused by miscommunication during parts orders, leaving us without key components. To adapt, we redesigned the robot using surplus popsicle sticks, L-brackets, and two servo motors. The final design used rear-mounted servos to lower arms that quickly collected cubes as the robot moved forward, allowing us to complete the final milestone. For final improvements, we optimized the code so the robot gathered cubes in the center before moving to a corner, added front popsicle sticks to prevent cubes from getting stuck underneath, and created backup components in case of damage during competition.

3. Competition Analysis

During the competition, our robot performed below expectations, winning two matches and losing three. Although the program generally worked as intended, we faced several unexpected issues. The main problem was that the collection

mechanism was structurally weak and frequently broke after colliding with opponent robots. We also experienced a malfunction where the robot would spin and stop unexpectedly. Eventually, we discovered that a loose wire connecting the QTI sensor to the Arduino had disrupted the robot's functionality and contributed to three losses. After repairing the wire, the robot performed correctly and won the final round. The robot's greatest strengths were its efficient code execution and strategic movement, which allowed it to reach the center and collect cubes before opponents. However, weak wiring and the fragile collection mechanism were major weaknesses, as damage often caused the robot to lose collected cubes and reduced its chances of winning.

4. Conclusions

Overall, the robot's biggest weakness was the fragility of its components. Loose wires and breakable arms could likely have been avoided with stronger materials, a sturdier structure, or better preparation during construction. Better coordination of our initial materials order may also have allowed us to complete our original design, improving reliability and competition performance. For future students, we recommend starting early and being proactive with design and construction, since early preparation provides more time to identify and fix problems.

5. Appendix A(Bill of Materials)

4 x Motor Mount for TT Gearbox DC Motors - L-bracket Type (Total: \$6.00)

1 x Rotary Shaft (Total: \$4.42)

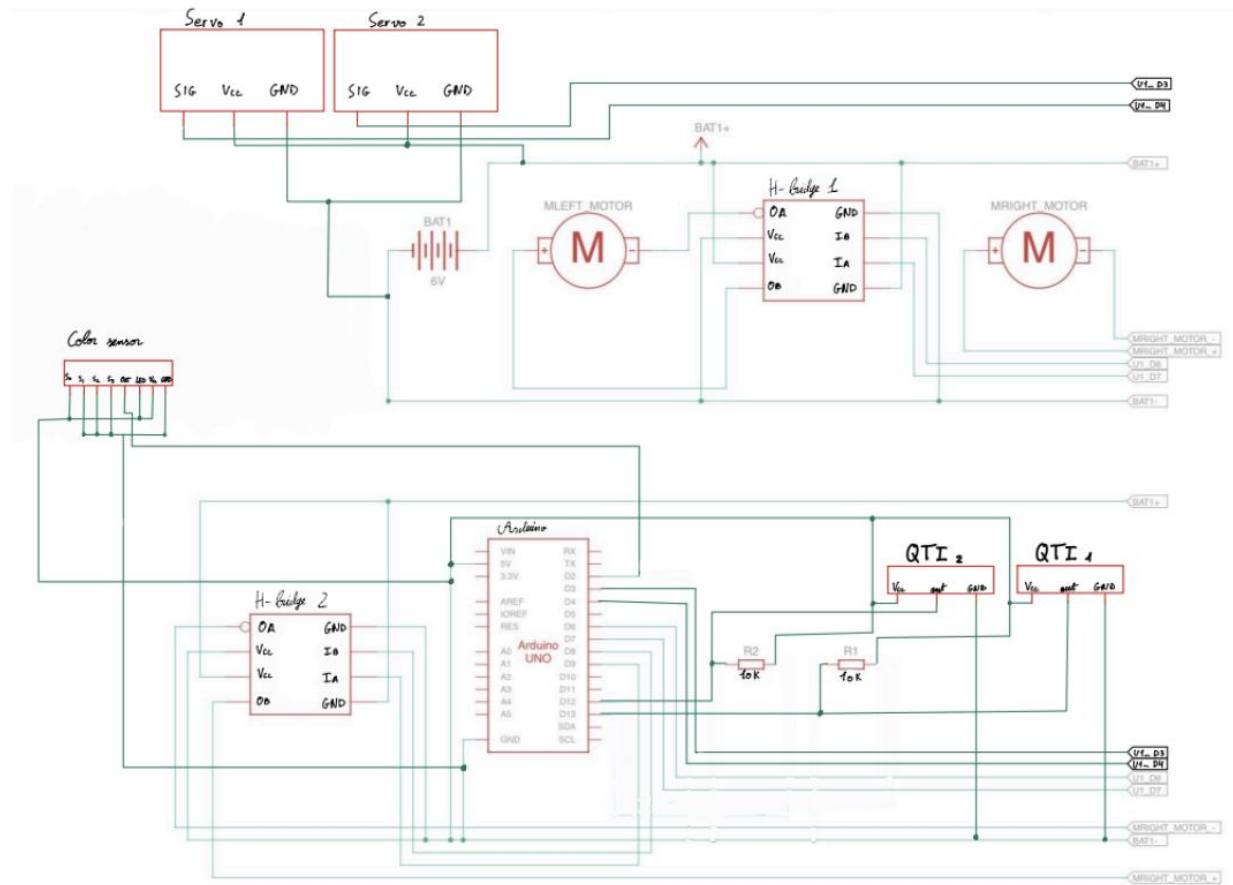
1 x Plastic Spur Gear (Total: \$6.16)

1 x 100 Pack Tongue Depressors (Total: \$7.31)

1 x 100 Pack Rubber Bands (Total: \$7.75)

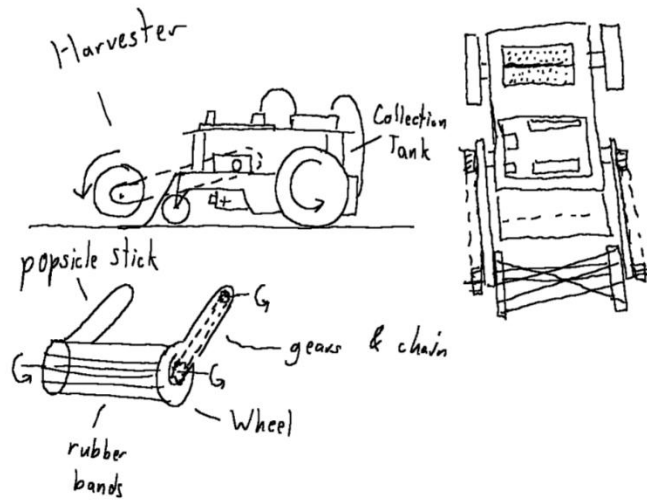
1 x Timing Belt (Total: \$13.07)

6. Appendix B (Circuit Diagram)

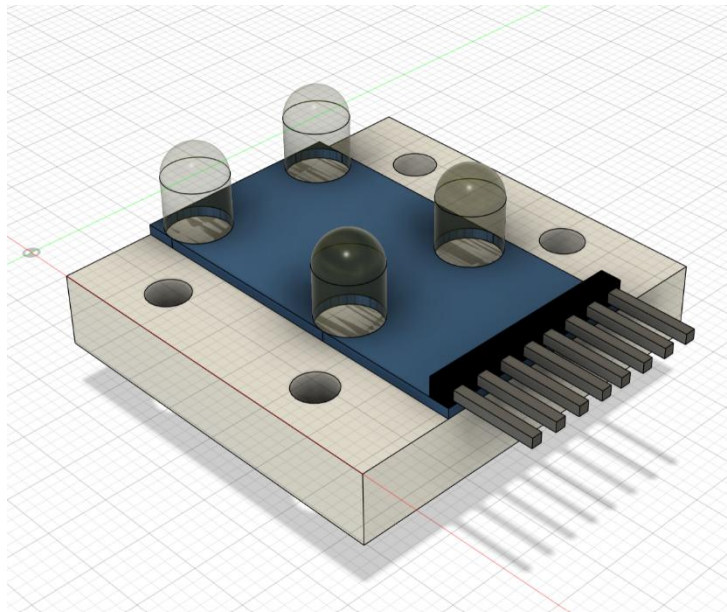


7. Appendix C(Drawings&CAD)

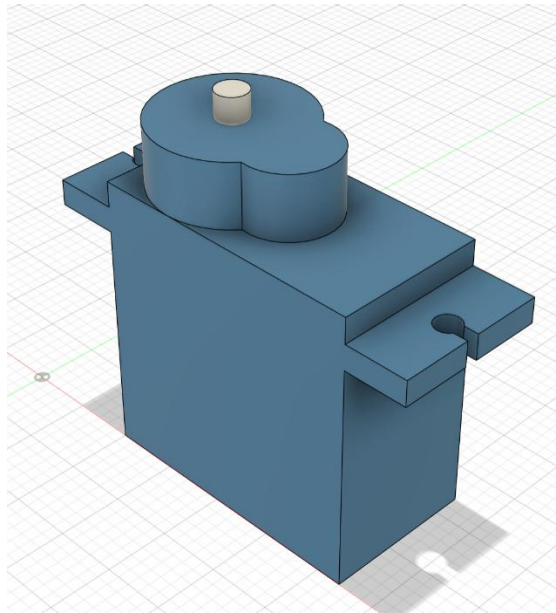
Final Sketch



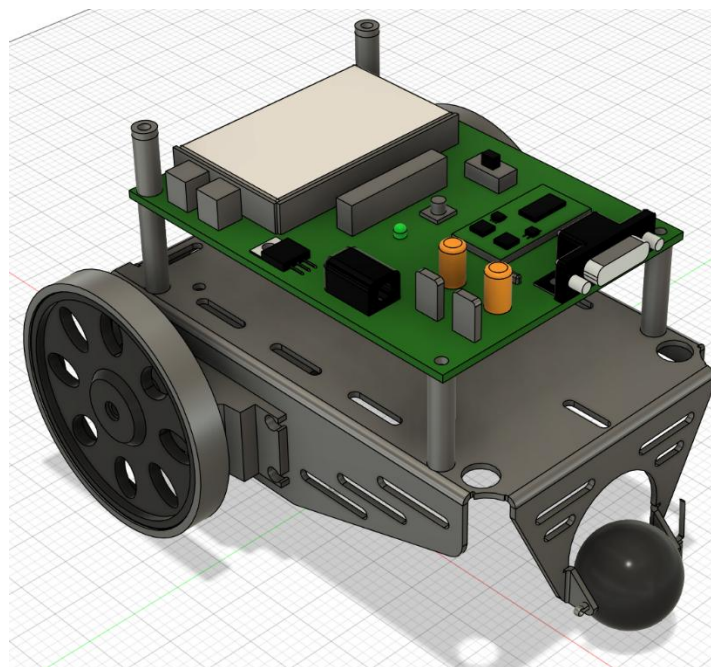
Initial Design Sketches



TCS3200 Color Sensor

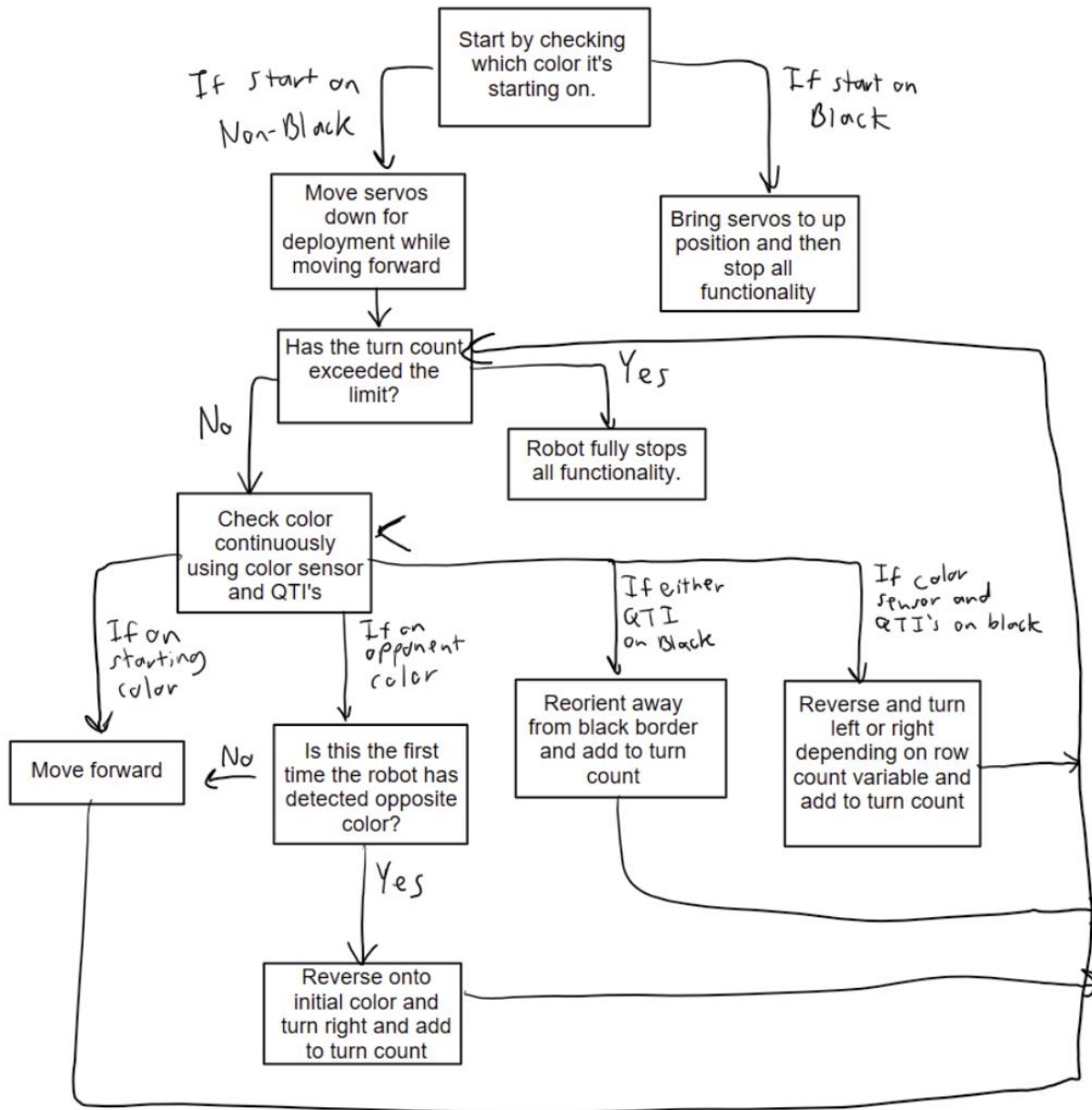


Positional Servo Motor



BOE Bot CAD (Provided)

8. Appendix D (Algorithm Flowchart)



9. Appendix E(Robot Arduino Code)

```
10. //global variables
11. volatile unsigned int period = 0;
12. volatile unsigned int timer = 0;
13. volatile unsigned int currentcolor = 0;
14. volatile unsigned int initialcolor = 0;
15. volatile unsigned int current = 0;
16.
17. ISR(PCINT2_vect) { //interrupt function to reset on rising edge
18.     if (PIND & (1 << 2)) {
19.         TCNT1 = 0;
20.     }
21.     else {
22.         timer = TCNT1;
23.     }
24. }
25.
26. // Initialize interrupt and timer for color detection
27. void initColor() {
28.     //define input
29.     DDRD &= ~(1 << PD2);    // PD2 is 0 so input
30.
31.     //initialize pin change interrupt
32.     PCICR |= (1 << PCIE2);
33.     PCMSK2 |= (1 << PCINT18);
34.
35.     //initialize timer
36.     TCCR1A = 0;
37.     TCCR1B = (1 << CS10);
38.     TCNT1 = 0;
39.
40. }
41.
42.
43. int getColor() { //get color function, returns 0 if black, 10 if
    yellow, 20 if blue
44.     PCMSK2 |= (1 << PD2);
45.     _delay_ms(10); //add short delay
46.     PCMSK2 &= ~(1 << PD2); //disable bit for interrupts until you call
    function again
47.
48.     // Timer is half the period, so full period = 2*timer each timer
    tick is 1/16 MHz = 0.0625 us,
49.     // so period in microseconds = 2 * timer * 0.0625 = timer / 8
```

```

50. period = timer / 8;
51.
52. // Determine the color, and record into "period"
53. if(period > 350) {
54.     period = 0; // black
55. }
56. else if (period < 150) {
57.     period = 10; //yellow
58. }
59. else {
60.     period = 20; // blue
61. }
62. return period; // Return "period"
63.
64. }
65.
66. int main(void){ // setup code that only runs once
67.     Serial.begin(9600); // start serial monitor for debugging
68.     sei(); // enable global interrupts
69.     initColor(); // call initColor function
70.
71.     DDRB = 0b00000011; // Initialize right motor
72.     DDRD = 0b11011000; // Initialize left motor and left/right servos
73.     period = getColor(); // Assign "period" to the current color
74.     initialcolor = period; // Use "period" to determine the starting
        color
75.     current = period; // Use "period" to set the current color
76.     Serial.println(initialcolor); //check initial color
77.
78.     // Main tracking variables for algorithm
79.     int rowCount = 0; //rowcount for knowing turn direction
80.     int servo = 0; //state of servo (up or deployed)
81.     int turncount = 0; //amount of turns before stopping
82.     int yell = 0; //variable for crossing color
83.
84.     Serial.print("Current Color: "); //show current color for
        debugging
85.     if (period == 0) {
86.         Serial.println("Black");
87.     } // Print the color
88.     else if (period == 10) {
89.         Serial.println("Yellow");
90.     }
91.     else {
92.         Serial.println("Blue");

```

```

93.  }
94.  if (current == 0){ //reset servo code on black and dont move
95.      for (int i = 0; i < 50; i++) { //positional servo
96.          PORTD |= (1 << PD4);
97.          _delay_ms(2);
98.          PORTD &= ~(1 << PD4);
99.          _delay_ms(18);
100.             servo = 1;
101.         }
102.         //360 servo code for up position
103.         for (int i = 0; i < 20; i++) { //integer determines how
            far servo rotates
104.
105.             PORTD |= (1 << PD3);
106.             _delay_ms(1); // forward direction
107.             PORTD &= ~(1 << PD3);
108.             _delay_ms(19);
109.         }
110.         for (int i = 0; i < 20; i++) {
111.             PORTD |= (1 << PD3);
112.             _delay_ms(1.5); // stop
113.             PORTD &= ~(1 << PD3);
114.             _delay_ms(18.5);
115.         }
116.     }
117.     else{ //down position for deployment
118.         drive_forward(); //move forward while dropping arms to
            save time
119.         //positional servo deploy code
120.         for (int i = 0; i < 70; i++) {
121.             PORTD |= (1 << PD4);
122.             _delay_ms(1);
123.             PORTD &= ~(1 << PD4);
124.             _delay_ms(19);
125.             servo = 0;
126.         }
127.         //360 servo deploy code
128.         for (int i = 0; i < 15; i++) { //integer determines how
            far servo rotates
129.             PORTD |= (1 << PD3);
130.             _delay_ms(2); // forward direction
131.             PORTD &= ~(1 << PD3);
132.             _delay_ms(18);
133.         }
134.         // STOP PD3 (keep sending stop pulses)

```

```

135.         for (int i = 0; i < 20; i++) {
136.             PORTD |= (1 << PD3);
137.             _delay_ms(1.5); // stop
138.             PORTD &= ~(1 << PD3);
139.             _delay_ms(18.5);
140.         }
141.         stop(20);
142.         drive_forward_delay(600); //immediately go to center (600
is 5.25in)
143.         while(1){ // code that loops forever
144.             current = getColor(); //check color
145.             //black border check first
146.             if (turncount <= 6){ // amount of turns before stopping
147.                 if (current == 0 && (PINB & (1 << 5)) && (PINB & (1 <<
4))) { //all sensors on black
148.                     Serial.println("TURN");
149.                     stop(100);
150.                     drive_backward_delay(200); //reverse back to color
151.                     if (rowCount % 2 == 0) { //if even row turn right
152.                         drive_right_delay(450);
153.                         drive_forward_delay(200);
154.                         drive_right_delay(450);
155.                         turncount = turncount + 1;
156.                     }
157.                     else{ //if odd row turn left
158.                         drive_left_delay(450);
159.                         drive_forward_delay(600);
160.                         drive_left_delay(450);
161.                         turncount = turncount + 1;
162.                     }
163.                     rowCount = rowCount + 1;
164.                     Serial.println("rowCount");
165.                 }
166.                 else if (PINB & (1 << 5)){ //left qti on black,
reorient and add turn count
167.                     drive_right_delay(250);
168.                     turncount = turncount + 1;
169.                 }
170.                 else if (PINB & (1 << 4)){ //right qti on black,
reorient and add turn count
171.                     drive_left_delay(250);
172.                     turncount = turncount + 1;
173.                 }
174.                 else if (current == initialcolor || yell==1){
//sweeping but other color has been passed so now can go past it

```

```

175.         drive_forward();
176.         Serial.println("FORWARD");
177.     }
178.     else if (current != initialcolor && yell == 0){//first
        time hit opponent color, go to side
179.         drive_backward_delay(200); //reverse back to color
180.         drive_right_delay(450); //drive right
181.         Serial.println("Reorient");
182.         turncount = turncount + 1;
183.         yell = yell +1; //yell was 0 now 1 for rest of round
        meaning can cross color
184.     }
185. }
186. else{
187.     stop(50000); //after finishing turns, stop
188. }
189. }
190. }
191. }
192.
193. //movement functions
194.
195. // option 1: movement functions with no delay
196. void drive_forward(){
197.     PORTB = 0b00000001; // Move "right" motor forward
198.     PORTD = 0b10000000; // Move "left" motor forward
199.
200.     // Serial.println("I am moving forward");
201. }
202.
203. void drive_backward(){
204.     PORTB = 0b00000010; // Move "right" motor backwards
205.     PORTD = 0b01000000; // Move "left" motor backwards
206.
207.     // Serial.println("I am moving backwards");
208. }
209.
210. void drive_right(){
211.     PORTB = 0b00000010; // Move "right" motor backwards
212.     PORTD = 0b10000000; // Move "left" motor forward
213.
214.     // Serial.println("I am turning right");
215. }
216.
217. void drive_left(){

```

```
218.     PORTB = 0b00000001; // Move "right" motor forward
219.     PORTD = 0b00000000; // Move "left" motor backwards
220.
221.     // Serial.println("I am turning left");
222. }
223.
224. // option 2: movement functions with built-in delay
225. void drive_forward_delay(int t){
226.     PORTB = 0b00000001; // Move "right" motor forward
227.     PORTD = 0b10000000; // Move "left" motor forward
228.
229.     _delay_ms(t); // wait for t milliseconds
230. }
231.
232. void drive_backward_delay(int t){
233.     PORTB = 0b00000010; // Move "right" motor backwards
234.     PORTD = 0b01000000; // Move "left" motor backwards
235.
236.     _delay_ms(t); // wait for t milliseconds
237. }
238.
239. void drive_right_delay(int t){
240.     PORTB = 0b00000010; // Move "right" motor backwards
241.     PORTD = 0b10000000; // Move "left" motor forward
242.
243.     _delay_ms(t); // wait for t milliseconds
244. }
245.
246. void drive_left_delay(int t){
247.     PORTB = 0b00000001; // Move "right" motor forward
248.     PORTD = 0b01000000; // Move "left" motor backwards
249.
250.     _delay_ms(t); // wait for t milliseconds
251. }
252.
253. // Stop the robot for a set amount of time
254. void stop(int t){
255.     PORTB = 0b00000011; // Stop "right" motor
256.     PORTD = 0b11000000; // Stop "left" motor
257.
258.     _delay_ms(t);
259. }
```